

Cifar-10 Classification using Deep Convolutional Neural Network

Raveen Doon
Netaji Subhash Institute of Technology
Delhi, India
raveend06@gmail.com

Dr. Tarun Kumar Rawat
Netaji Subhash Institute of Technology
Delhi, India
tarundsp@gmail.com

Dr. Shweta Gautam
Netaji Subhash Institute of Technology
Delhi, India
Shwetauiet11@gmail.com

Abstract—Deep learning models such as convolution neural networks have been successful in image classification and object detection tasks. Cifar-10 dataset is used in this paper to benchmark our deep learning model. Various function optimization methods such as Adam, RMS along with various regularization techniques are used to get good accuracy on image classification task.

Keywords—Convolution, Max-pooling, CIFAR-10

I. INTRODUCTION

Convolutional neural networks are widely used image for classification and object recognition tasks. Convolution layer is responsible for extracting important features with the help of filters or feature extractor. A deep neural network has been implemented to perform image classification on CIFAR-10 dataset. Various hyperparameters need to be tuned to extract best performance from the network. To avoid overfitting, we have tried to reduce the model complexity by using dropout and tried to increase data size by using data augmentation. The choice of function optimizer is very important since it directly affects convergence speed. Adam optimizer has been used in the network. ReLU activation function is used throughout the network except at the output layer where Softmax activation function is used. The output labels are in the form of integers and need to be changed to categorical using one hot encoding. During training each forward pass results in an output value which is compared with the actual output and loss function is computed. Cross-entropy loss function is used along with Softmax activation at the output and the gradient of the loss function are propagated back into the network to optimize the weights. This process is repeated for fixed number of epochs to get best possible accuracy.

II. THE DATASET

CIFAR-10 dataset is used for training the network. The dataset consists of 60000, 32x32 color images divided into 10 classes. Each class has 6000 images. The images are further divided into training and test set with each set having 50000 and 10000 images respectively. The CIFAR-10 classes are: Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship, Truck.

III. THE ARCHITECTURE

The architecture of the convolutional neural network is summarized in figure 1. The network has 6 convolution layers, 3 pooling layers and 3 fully connected layers. Below is a discussion of some of the important features of the network architecture

A. Leaky ReLu Activation Function

Leaky ReLu [8][9] is a nonlinear Activation function. It is an important part of our neural network architecture since it introduces non-linearity to the network. In the absence of nonlinear activation function deep neural net would behave like a single layered network since summing these layers would just give us another linear function. Neural networks are considered universal function approximators. The use of nonlinear activation function and hidden layers gives it an ability to approximate complex functions. There is a choice of using various activation functions like, Sigmoid, Tanh, ReLu, Leaky ReLu etc. Sigmoid function has non-zero mean and during backpropagation through the network, gradients in neurons whose output is near 0 or 1 are nearly 0.

Hence these neurons are called saturated neurons. The weights of these neurons are not updated, and weights of the neurons connected to these are updated slowly. This is known as vanishing gradient. Moreover, these activation functions are computationally expensive. Hyperbolic tangent activation function also suffers from vanishing gradient but is slightly better than sigmoid activation function since it is zero centered and has 0 mean. These saturating nonlinear functions lead to slower training with gradient descent as compared to non-saturating nonlinear functions like ReLu and leaky ReLu.

ReLu activation function makes the network converge much faster as it does not saturate (when $x > 0$) and is computationally efficient. It is defined as, $f(x) = \max(0, x)$ but it suffers from a drawback, when $x < 0$, during forward pass neurons remain inactive and weights are not updated during backpropagation. As a result, the network does not learn. Hence, we use leaky ReLu [8] which is defined as

$$f(x) = \max(\alpha x, x) \quad (1)$$

Here, since for $x < 0$ the function has a small finite slope the derivation is no longer 0 for $x < 0$ and weights are updated during backpropagation.

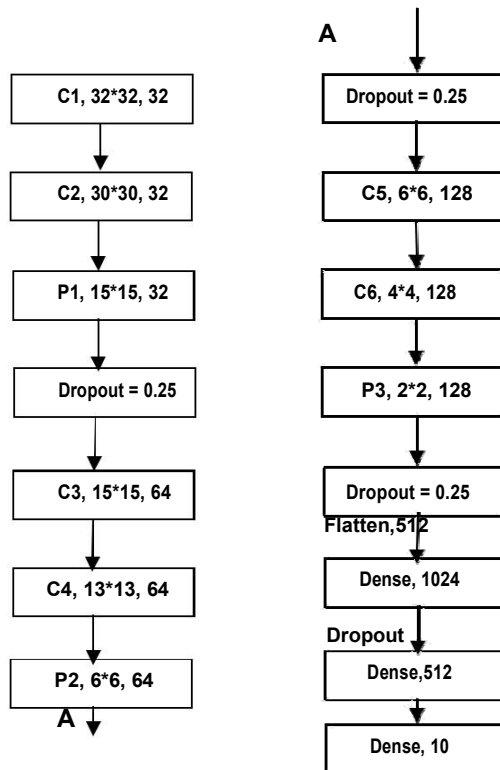


Fig 1: ConvNet architecture

B. Convolutional layers

The training data used for training the deep neural network [9] consists of images of objects from 10 different classes. Pixels in image data form patterns and rearranging the rows or columns of images would break this pattern. Convolutional neural networks only capture local spatial patterns in data. Convolutional nets are great at finding patterns and using them to classify images.

Convolution layer consist of input image and a stack of filters/features. A 2D convolution is performed between image array and filter resulting in a stack of filtered image arrays at the output. The filtered images consist of numbers representing how well the filter matched that part of the image. This becomes the map of where the feature occurs. This act of convolving an image with a bunch of filters and creating a stack of filtered images is called convolution layer. The input to first convolution layer in our network is a single image and output is a stack of 32 filtered images. The network has 6 convolution layers with pooling layers stacked in between.

C. Batch Normalization

First layer of the deep network has training data as input which does not change during training, but hidden layers take the output of previous layers as input, which changes as the previous layers weights are optimized during backpropagation. Thus, we can view the input to each layer as a shifting dataset of hidden activations during training [9].

Covariate shift in the input data results in retraining of network. A shifting input causes convergence of weights during training to become slower. Batch normalization reduces the amount by which the distribution of these hidden units shifts around. The mean and variance of the data input to hidden layers does not change much after applying batch normalization layer.

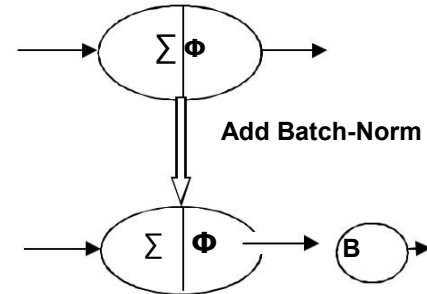


Fig 2: Batch normalization layer

We add an additional layer that normalizes in batch wise fashion. Batch-norm parameters become optimized during training. Batch-norm has the effect of increasing the training speed.

The i^{th} unit contains in Batch-norm layer contains two parameters β_i and γ_i that need to be learned.

Steps in batch normalization [9]:

1. Normalize over batch of m instances for i^{th} unit.

$$\mu_i = \frac{1}{m} \sum_{j=1}^m x_{ij}, \text{ for all } i, \text{ Batch mean}$$

$$\sigma_i^2 = \frac{1}{m} \sum_{j=1}^m (x_{ij} - \mu_i)^2, \text{ for all } i, \text{ Batch variance}$$

$$\hat{x}_{ij} = \frac{x_{ij} - \mu_i}{\sigma_i}, \text{ for } i, r, \text{ normalized batch instances}$$

$$y_i = \hat{x}_{ir} * \gamma_i + \beta_i, \text{ for all } i, r \text{ (Scale with learnable parameters)}$$

Most activations will be near 0 (near linear region), γ and β provide the ability to each layer to decide how much non-linearity they want.

D. Max Pooling layer

Max pooling layers [1] are applied in the network architecture after every two convolution layers. There are 3 pooling layers in total with size of 2x2. These pooling layers shrink the image stack by retaining the maximum value. Max Pooling is applied on entire stack of filtered images to get smaller stack of filtered images.

E. Loss Function

The activation function used in the convolution layers and hidden layers is ReLU which works well for those layers. The output layers need to classify images into 10 different classes and uses Softmax activation function. Softmax function forces output of the neural network to sum to 1 so that they can represent a probability distribution across a bunch of discrete mutually exclusive alternatives.

$$y_i = \sum \in \quad (2)$$

sum of y_i must be 1.

The way the units in a Softmax group work is they receive some total input they have accumulated from the layer below. Which is z_i for the i^{th} unit, but y_i does not just depend on z_i but also on z_j , accumulated by other nodes.

Here, y_i represents the probability distribution over mutually exclusive alternatives just by using Softmax equation. The right cost function to use with Softmax activation is Cross-entropy [9].

$$C = -\sum \quad (3)$$

t_j = Target value

y_j = Actual value

C has a very big gradient when the target value is 1 and the output is almost 0. The steepness of — exactly balances the flatness of.

$$\text{—} = \text{—} = \text{—} \quad (4)$$

when the difference between y_i and t_i is large the slope is 1 or -1, but when the difference is small, slope becomes smaller.

IV. REDUCING OVERFITTING

To check overfitting, the data is splitted into two portions, validation data and training data. The error is plotted on the training and testing stats as the model is trained. A large difference in the error graph on the training and validation dataset indicated overfitting. To reduce overfitting, either reduce model complexity using regularization or increase dataset by methods such as data augmentation. One way to implement regularization is to add a term to our cost function that penalizes overly complex models. Overfitted neural network is unable to generalize well. It learns the features of training data well but when we input test data which deviates from training data, the system doesn't give good results. A large dataset gives more diversity to our model. When the performance on unseen data is not as good as performance on unseen data, there is a generalization gap between performance to train and test data.

Table 1: Classification of loss curves [9]

Training loss	Validation loss	
High	High	Underfitting
Low	High	Overfitting
Low	Low	Good fit
High	low	Unlikely

An underfitted data has low variance but high bias, whereas overfitted model may have low bias but high variance. Our model initially had a low training loss but high validation loss indicating Overfitting. The solution to overfitting is to reduce capacity by regularization and Dropout and increase data by data augmentation.

A. Regularization

Encourages models to have a preference towards simpler models without making a part of the hypothesis space completely unavailable. These techniques intend to reduce the risk of overfitting without increasing bias significantly.

In regularization,

Loss $\leftarrow$ Loss + (Weight penalty)

Weight decay term puts a constraint on the model by penalizing large weights.

$$\sum f(y_i - y_i') + \alpha \sum \quad (5)$$

α represents regularization strength and is a hyperparameter used to tune our model [9].

B. Dropout

Dropout [7] randomly drops out neurons with some probability during training. The training algorithm uses random subset of the network at every iteration. This approach encourages neurons to learn useful features on their own without relying on other neurons. Once trained the entire network is used for inference. To prevent overfitting we have two options, we can either reduce the effective capacity of the network by using methods such as dropout or we can increase the amount of training data by generating modified versions of the samples we already have using data augmentation.

V. FUNCTION OPTIMIZATION

In Function optimization we try to minimize the loss function by finding the optimum parameters of the network. Most common method to find the minimum is gradient descent. This is a first order method which finds the slope of the tangent of the graph of the loss function w.r.t weights. Gradient descent optimization problem [9]:

$$L(w) = \sum (; ,) \rightarrow \min w$$

W^0 = initialization

Weights update equation for gradient descent is:

$$W^t = w^{t-1} - \alpha () \quad (6)$$

In this method l gradients are computed on each step and hence this method is a bit slower since l can be very large. Gradient descent is infeasible for large training sets.

A. Stochastic gradient descent

This is like gradient descent except the weights are updated only on one example instead of the entire set. But this method leads to noisy updates and convergence is not smooth. A better alternative to SGD method is mini-batch gradient method where weights are updated on a mini-batch which is a subset of the entire set. Here we have m points to estimate gradient instead of 1 point in SGD. Stochastic and mini-batch descents use gradient approximations to speedup computations, but suitable learning rate is hard to select. For difficult functions gradient descent oscillates and it takes many iterations for it to converge to optimum weights [9].

Improvement over gradient descent,

W^0 – initialization.

While true:

$i_1, \dots, i_m = \text{random indices between 1 and } l$

$$g_t = -\sum (\quad , \quad) \quad (7)$$

$$w^t = w^{t-1} - \alpha g_t \quad (8)$$

if $\|w^t - w^{t-1}\| < \epsilon$ then break

Momentum

In this method we maintain an additional vector h at every iteration [9].

$$h_t = \beta h_{t-1} + \alpha g_t \quad (9)$$

$$w^t = w^{t-1} - h_t \quad (10)$$

Above equation corresponds to gradient step in the direction of $-h_t$. h_t is weighted sum of gradient from all previous iterations and present iteration as well. The step is taken in the direction determined not only by the current gradient but all the previous gradients as well. This prevents fluctuations and oscillations as seen in SGD.

Nesterov momentum

$$h_t = \beta h_{t-1} + \alpha (\quad - \beta h_{t-1}) \quad (11)$$

$$w^t = w^{t-1} - h_t \quad (12)$$

This has better convergence than momentum. Momentum and Nesterov momentum work better but we still need to choose learning rate manually in above methods. Methods discussed below can change learning rate adaptively.

B. RMS prop

$$= \beta + (1 - \beta) \quad (13)$$

$$= - \quad (14)$$

g_{ij} represents gradient w.r.t j^{th} parameter, we get separate learning rate for each dimension. Here learning rate adapts to latest gradient steps.

C. Adam [2]

$$= \frac{(\quad)}{\quad} \quad (15)$$

$$= - \quad \in \quad (16)$$

VI. RESULTS

The final model was trained on the CIFAR-10 data set. The dataset is divided into training data, comprising of 50000 images and testing data, consisting of 10000 images. The network was trained for 100 epochs. The training data was further divided into 196 batches. Therefore 1 epoch is completed when the network is trained over all the 196 batches with each batch consisting of 256 images. The results of training for 100 epochs are summarized in the Table 2

Table 2: Accuracy obtained for different Dataset

Data	Epochs	Loss	Accuracy
Training data	100	0.2789	90 %
Validation data	100	0.3830	87.57 %
Testing data	1	0.3830	87.57 %

Below figure shows the comparison of convergence rate of Stochastic gradient descent with Adam. As can be inferred from the graph Adam converges to optimum solution much faster than Stochastic gradient descent.

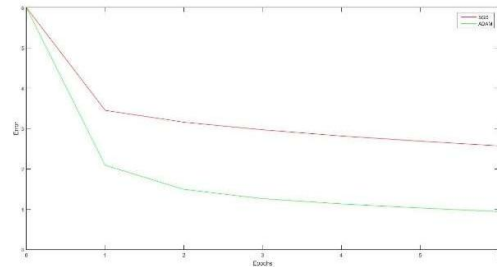


Fig 3 Error comparison for SGD and Adam

Fig 4 shows the results of training before applying data augmentation, the network is overfitted as evident from difference between training and validation curves

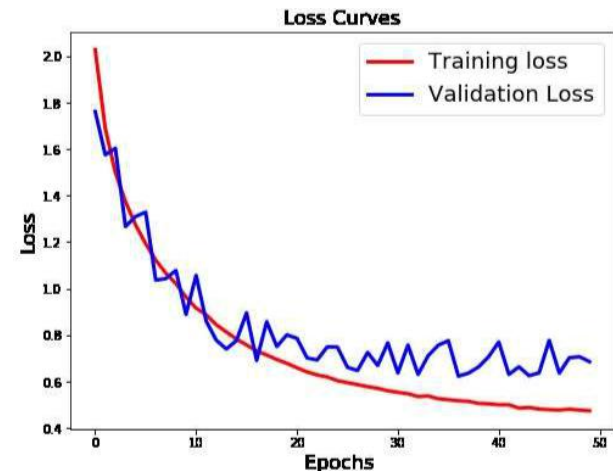


Fig 4 Loss vs Epochs curve for slightly overfitted model

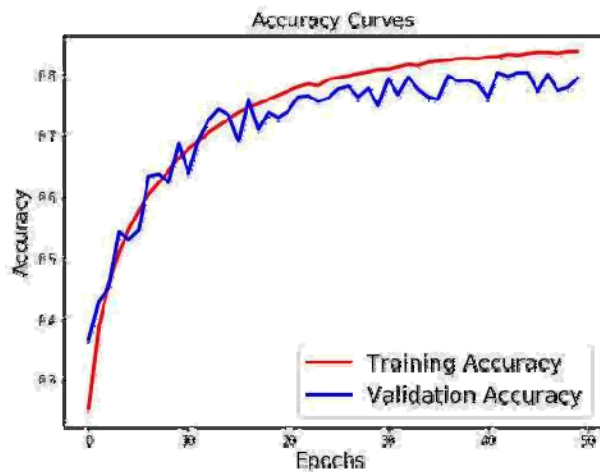


Fig 5 Accuracy vs epochs curve for slightly overfitted model
The fig below shows the effect of applying data augmentation to our final network. The overfitting is avoided this way.

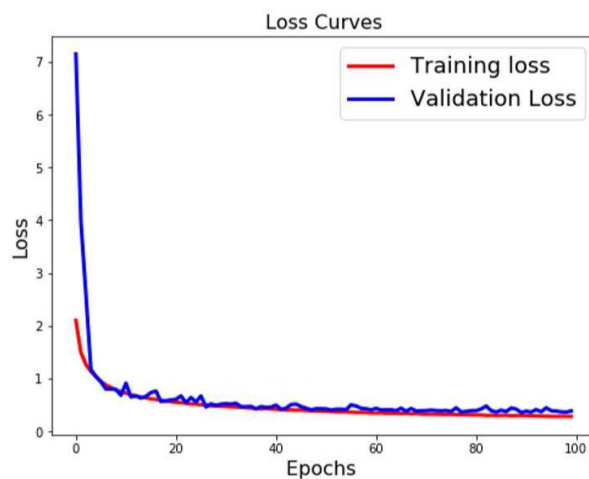


Fig 6 Loss vs Epochs curve for model after removing overfitting

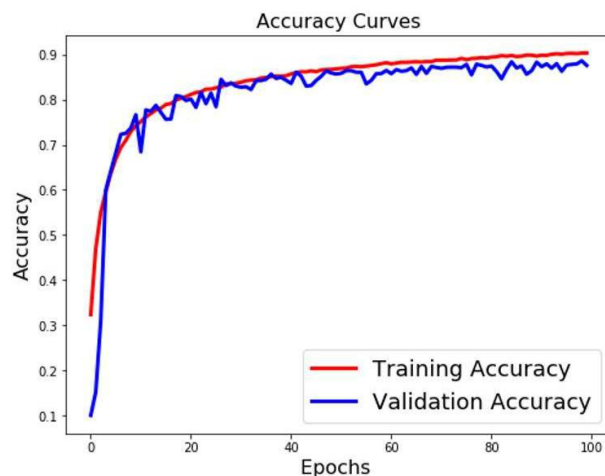


Fig 7 Accuracy vs Epochs curve for model after removing overfitting

VII. CONCLUSION

In this paper we have implemented a Convolutional neural network to classify images into 10 categories. CIFAR-10 dataset has been used as a benchmark to train the network. The Adam optimization technique to update weights gives us the best accuracy in terms of image classification. The model overfitting is reduced by using regularization and dropout methods. Using deep network architecture, we were able to achieve 90% accuracy on the training data and 87.57 % accuracy on the test set.

VIII. REFERENCES

1. Goodfellow, I., Bengio, Y. and Courville, A. (n.d.). Deep learning. MIT Press, pp.164-365.
2. Kingma, D., & Ba, J. (2014). Adam: A method for stochastic optimization. arXiv preprint arXiv:1412.6980.
3. Zeiler, M. D.(2012). ADADELTA: an adaptive learning rate method. arXiv preprint arXiv:1212.5701.
4. E. Hinton, et al." Improving neural networks by preventing coadaptation of feature detectors",2012/07/03, arXiv preprint.
5. LecunY, Bottou L, Bengio Y, et al. Gradient-based learning applied to document recognition[J]. Proceedings of the IEEE, 1998, 86(11):2278-2324.
6. M. D. Zeiler and R. Fergus, "Visualizing and understanding convolutional networks," in ECCV, 2014.
7. S. Wang and C. Manning, "Fast dropout training," in ICML, 2013.
8. V. Nair and G. E. Hinton, "Rectified linear units improve restricted boltzmann machines," in ICML, 2010, pp. 807–814.
9. deeplearning.ai. (2018). Home - deeplearning.ai. [online] Available at: <https://www.deeplearning.ai/> [Accessed 15 Jul. 2018].